Project Checkpoint: Neural Cleanse Reproduction and Trigger-Size Stress Test (Topic M07)

Team: Shivraj Nath (sdn55), Roger Mendez (rtm141)
Repository: https://github.com/shivrajjjjn/BackDoorInfoNetwork
Branch: shivraj/gpu-port
Team note: Daniel Kang left the group. Per the instructor, we are only covering the parts slated in our project proposal.

FALLBACK TRACE

logs/baseline_gtsrb_infected_gpu_run1.txt (commit 72e2f4e), with its detection output logs/baseline_gtsrb_infected_gpu_mad1.txt (commit 3abedc0). If our later snapshot is missing or invalid, please use these.

Status: The Neural Cleanse baseline runs end to end on the authors' infected GTSRB model and passes our sanity check. MNIST backdoor training and the trigger-size sweep are in progress.

Checklist:

1. A runnable baseline or a harness that supports your experiments, with instructions for running it.
2. One checkable trace or failure log. Name the sanity check you ran and explain the result.
3. Your bounded-contribution plan: what you will add or change beyond the baseline.
4. Evidence showing each team member's work so far.
5. Your updated experimental protocol and logs.

1. Runnable Baseline

We reproduce Neural Cleanse (Wang et al., IEEE S&P 2019) using the authors' code (github.com/bolunwang/backdoor, MIT license) on their provided infected GTSRB model (white 4-pixel patch, bottom-right, target label 33). We ported the code from TensorFlow 1 to TensorFlow 2 so it runs on an RTX 5060, since it would be unreasonably long trying to use a CPU.

How to run (full instructions in README_PROJECT.md in the repo):
1. Create the conda environment and install the packages listed in README.md and requirements-gpu.txt (Python 3.11, TensorFlow 2.21.0).
2. python gtsrb_visualize_example.py (reverse-engineers a trigger for each of the 43 labels; about 5 minutes on our GPU)
3. python mad_outlier_detection.py (MAD anomaly detection on the reversed triggers)

Hardware: Ryzen 5 5600X, RTX 5060. A first attempt in the original environment on CPU took about 86s per label (about 1 hour projected), so we ported to GPU.
Code changes to the authors' code are listed in README_PROJECT.md.

2. Checkable trace and sanity check

Trace: logs/baseline_gtsrb_infected_gpu_run1.txt and
logs/baseline_gtsrb_infected_gpu_mad1.txt

Sanity check: Known-target recovery. The provided model is infected with a trigger that targets label 33. If our reproduction is correct, detection should flag label 33 (and only that label), with an anomaly index above 2 and a trigger L1 norm far below the median.

Result: PASSED.
- 43 labels analyzed; only label 33 was flagged
- Anomaly index: 2.68 (threshold 2)
- Label 33 L1 norm: 11.92, versus a median of 60.68 and a MAD of 18.20

Compared with the authors' reported values (index 3.65, flagged norm 16.12, median 64.47, MAD 13.24), our anomaly index is lower. The main reason is that the spread of the other labels' norms (MAD) is larger in our run. The detection still passes, with a smaller margin above 2.

Earlier partial check (CPU run, stopped early): label 33 mask norm 17.34 versus label 0 mask norm 54.70.

Failure logs kept (porting errors we fixed): failure_gpu_run0_set_random_seed.txt, failure_gpu_run1_adam_get_updates.txt, failure_gpu_run2_scipy_missing.txt.

3. Bounded-contribution plan

Our bounded contribution adds a stress test with a negative control, specifically scoped to single-patch triggers. This plan includes

- Trigger-size sweep: With a fixed target label and poison rate, the trigger area can be varied to find where the true target label's anomaly index falls below the detection threshold of 2.
- Clean-model control: Run the Neural Cleanse detector on identically trained clean models to measure the false-positive rate.
- Mitigation check: On correctly flagged models, apply the filtering step and report the clean accuracy and attack success rate before and after mitigation.
- Out of scope: Multi-patch triggers, adaptive attacks, and other detectors are explicitly out of scope.

4. Evidence of each member's work

Shivraj Nath (sdn55)	Roger Mendez (rtm141)
Reproduced the baseline and MAD detection Ran sanity check Ported the authors code to work on a GPU	Wrote the BadNets trigger injection and training script for MNIST, the clean-model control script, and the automated sweep runner to orchestrate the experiments.
Evidence is in commits and logs under name	Evidence is in commits and logs under name
Status: Baseline is done.	Status: MNIST backdoor training and sweep scripts are implemented and running.

5. Updated protocol and logs

Baseline protocol (done): GTSRB infected model, authors' default optimization parameters, DEVICE=0, results in results_gpu/. Measured time: 285 s for all 43 labels on the RTX 5060 (about 6.6 s per label).

MNIST sweep protocol:

Fixed Parameters: 10% poison rate, target label 0, fixed epochs, and fixed learning rate.

Sweep Variables: 4 trigger sizes (e.g., small, medium, and intermediate sizes) across 3 random seeds.

Clean Control: 3 clean models trained with a 0% poison rate.

GTSRB Scope: Testing is limited strictly to the provided infected model.

Time & GPU-Hours Estimate: Based on our baseline measurement of 285 seconds (~5 minutes) per detection run on the RTX 5060, this scaled-down MNIST sweep will complete well under our original 10-20 GPU-hour budget.

Logs: The sweep outputs are appended to results/sweep.csv in the repository.

Logs: Present in repository.